

NAME: PRIYADHARSHINI K S

REG NO: 192511248

COURSE: COMPUTER ARCHITECTURE

CODE: CSA1204

Q1. Design and Simulation of Register Transfer, Arithmetic and Logic Operations

Aim:

To design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as **addition, subtraction, AND, and OR**, and to analyze the role of the ALU during processing.

Introduction

A register is a small, high-speed storage location inside the processor used to temporarily store data and instructions. **Register Transfer** refers to moving data from one register to another.

The **Arithmetic Logic Unit (ALU)** is the main processing unit responsible for performing arithmetic and logical operations. In this experiment, multiple registers are connected to an ALU to demonstrate how data is transferred and processed.

R1 = 10

R2 = 5

Register Transfer

R3 = R1

print("Register Transfer: R1 -> R3")

print("R3 =", R3)

Arithmetic Operations

```

print("\nArithmetic Operations")

print("Addition:", R1 + R2)

print("Subtraction:", R1 - R2)


# Logical Operations

R1_binary = 0b1010

R2_binary = 0b1100


print("\nLogical Operations")

print("AND:", bin(R1_binary & R2_binary))

print("OR :", bin(R1_binary | R2_binary))

```

```

Register Transfer:
R1 = 10
R2 = 5
R3 <- R1 = 10

Arithmetic Operations:
R3 <- R1 + R2 = 15
R3 <- R1 - R2 = 5

Logical Operations:
R3 <- R1 AND R2 = 1000
R3 <- R1 OR R2 = 1110

...Program finished with exit code 0
Press ENTER to exit console.

```

Result and Analysis

The designed system successfully demonstrates register-to-register data transfer and ALU-based processing. Addition and subtraction are performed as arithmetic operations, while AND and OR are performed as logical operations.

The experiment shows that proper control signals are required to select registers, transfer data and determine the ALU operation. Multiple registers allow intermediate values to be stored and transferred efficiently.

Q2. Single-Bus and Multi-Bus Organization

Aim

To create a model of a computer system using **single-bus and multi-bus organization**, perform data transfer between registers, memory and I/O, and compare their time and efficiency.

Introduction

A bus is a communication pathway used to transfer data between different components of a computer system. Registers, memory and I/O devices communicate with each other through buses.

Two common bus organizations are:

- Single-bus organization
- Multi-bus organization

Single-Bus and Multi-Bus Organization

R1 = 20

R2 = 10

```
print("SINGLE-BUS ORGANIZATION")
```

```
print("-----")
```

```
print("R1 =", R1)
```

```
print("R2 =", R2)
```

```
print("Step 1: R1 -> Common Bus")
```

```
print("Step 2: R2 -> Common Bus")
```

```
result_single = R1 + R2
```

```
print("ALU Operation: R1 + R2")
```

```
print("Result =", result_single)
```

```
print("\nMULTI-BUS ORGANIZATION")
```

```
print("-----")
```

```
print("R1 -> Bus A")
```

```
print("R2 -> Bus B")
```

```
result_multi = R1 + R2
```

```
print("ALU Operation: R1 + R2")
```

```
print("Result =", result_multi)
```

```
print("\nCOMPARISON")
```

```
print("Single-Bus : Sequential data transfer")
```

```
print("Multi-Bus : Parallel data transfer")
```

```

SINGLE-BUS ORGANIZATION
-----
R1 = 20
R2 = 10
Step 1: R1 -> Common Bus
Step 2: R2 -> Common Bus
ALU Operation: R1 + R2
Result = 30

MULTI-BUS ORGANIZATION
-----
R1 -> Bus A
R2 -> Bus B
ALU Operation: R1 + R2
Result = 30

COMPARISON
Single-Bus : Sequential data transfer
Multi-Bus  : Parallel data transfer

...Program finished with exit code 0
Press ENTER to exit console.

```

Result

The single-bus and multi-bus systems were modeled successfully. The multi-bus organization provides faster and more efficient data transfer because it supports greater parallelism.

Q3. Five-Stage Instruction Pipeline

Aim

To simulate a **5-stage instruction pipeline** consisting of Fetch, Decode, Execute, Memory and Write-back, and compare its performance with a non-pipelined processor using speedup and throughput.

Introduction

Pipelining is a technique used in processors to improve instruction execution speed. An instruction is divided into several stages, and different instructions can occupy different stages simultaneously.

```
# Five-Stage Instruction Pipeline
```

```
instructions = ["I1", "I2", "I3", "I4", "I5"]
```

```
stages = ["IF", "ID", "EX", "MEM", "WB"]
```

```
print("5-STAGE INSTRUCTION PIPELINE\n")
```

```
# Pipeline execution
```

```
for cycle in range(1, len(instructions) + len(stages)):
```

```
    print("Cycle", cycle, ":", end=" ")
```

```
    for i in range(len(instructions)):
```

```
        stage = cycle - i - 1
```

```
        if 0 <= stage < len(stages):
```

```
            print(instructions[i] + "-" + stages[stage], end=" ")
```

```
    print()
```

```
# Performance calculation
```

```
non_pipelined = len(instructions) * len(stages)
```

```
pipelined = len(instructions) + len(stages) - 1
```

```
speedup = non_pipelined / pipelined
```

```

print("\nPerformance Analysis")

print("Non-pipelined cycles =", non_pipelined)

print("Pipelined cycles   =", pipelined)

print("Speedup           =", round(speedup, 2))Result

```

```

5-STAGE INSTRUCTION PIPELINE

Cycle 1 : I1-IF
Cycle 2 : I1-ID  I2-IF
Cycle 3 : I1-EX  I2-ID  I3-IF
Cycle 4 : I1-MEM I2-EX  I3-ID  I4-IF
Cycle 5 : I1-WB  I2-MEM I3-EX  I4-ID  I5-IF
Cycle 6 : I2-WB  I3-MEM I4-EX  I5-ID
Cycle 7 : I3-WB  I4-MEM I5-EX
Cycle 8 : I4-WB  I5-MEM
Cycle 9 : I5-WB

Performance Analysis
Non-pipelined cycles = 25
Pipelined cycles     = 9
Speedup              = 2.78

...Program finished with exit code 0
Press ENTER to exit console.

```

The five-stage pipeline was successfully simulated. The pipelined processor required fewer cycles than the equivalent non-pipelined processor.

Q4. Pipeline Hazards and Hazard Resolution Techniques

Aim

To develop a pipeline execution scenario containing **data, control and structural hazards**, observe their effects and implement stalling, data forwarding and branch prediction techniques.

Introduction

A pipeline hazard is a situation that prevents the next instruction from executing in its normal pipeline cycle.

There are three major types:

1. Data Hazard
2. Control Hazard
3. Structural Hazard

Pipeline Hazards and Hazard Resolution

```
print("PIPELINE HAZARDS\n")
```

Data Hazard

```
print("1. DATA HAZARD")
```

```
print("I1: R1 <- R2 + R3")
```

```
print("I2: R4 <- R1 + R5")
```

```
print("Cause: I2 depends on the result of I1")
```

```
print("Solution: Data Forwarding")
```

Control Hazard

```
print("\n2. CONTROL HAZARD")
```

```
print("Instruction: BEQ R1, R2, TARGET")
```

```
print("Cause: Next instruction depends on branch decision")
```

```
print("Solution: Branch Prediction")
```

Structural Hazard

```
print("\n3. STRUCTURAL HAZARD")
```

```
print("Instruction Fetch and Data Access use the same resource")
```

```
print("Cause: Hardware resource conflict")
```

```
print("Solution: Stalling")
```

```
# Summary
```

```
print("\nHAZARD RESOLUTION SUMMARY")
```

```
print("Data Hazard    -> Forwarding")
```

```
print("Control Hazard -> Branch Prediction")
```

```
print("Structural Hazard -> Stalling")
```

PIPELINE HAZARDS

1. DATA HAZARD

```
I1: R1 <- R2 + R3
```

```
I2: R4 <- R1 + R5
```

```
Cause: I2 depends on the result of I1
```

```
Solution: Data Forwarding
```

2. CONTROL HAZARD

```
Instruction: BEQ R1, R2, TARGET
```

```
Cause: Next instruction depends on branch decision
```

```
Solution: Branch Prediction
```

3. STRUCTURAL HAZARD

```
Instruction Fetch and Data Access use the same resource
```

```
Cause: Hardware resource conflict
```

```
Solution: Stalling
```

HAZARD RESOLUTION SUMMARY

```
Data Hazard    -> Forwarding
```

```
Control Hazard -> Branch Prediction
```

```
Structural Hazard -> Stalling
```

```
...Program finished with exit code 0
```

```
Press ENTER to exit console.
```

Result

The three major pipeline hazards were identified and their effects on instruction execution were analyzed. Stalling, forwarding and branch prediction were applied as appropriate hazard-resolution techniques.

Q5. Superscalar, Out-of-Order, Speculative Execution and SMT

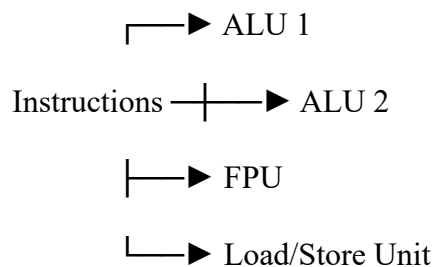
Aim

To analyze a processor supporting **superscalar, out-of-order and speculative execution**, and study hyper-threading and simultaneous multithreading (SMT) to evaluate instruction throughput and resource utilization.

Introduction

A superscalar processor can execute more than one instruction in a single clock cycle by using multiple execution units.

For example:



This allows independent instructions to execute in parallel.

Superscalar, Out-of-Order, Speculative Execution and SMT

```
instructions = [  
    ("I1", "ADD", "Waiting"),  
    ("I2", "SUB", "Ready"),  
    ("I3", "AND", "Ready"),  
    ("I4", "OR", "Ready")
```

```
]
```

```
print("SUPERSCALAR EXECUTION")
```

```
print("-----")
```

```
for instruction, operation, status in instructions:
```

```
    if status == "Ready":
```

```
        print(instruction, "->", operation, "executed")
```

```
print("\nOUT-OF-ORDER EXECUTION")
```

```
print("-----")
```

```
print("I1 is waiting for data")
```

```
print("I2 is ready -> Execute I2")
```

```
print("I3 is ready -> Execute I3")
```

```
print("I1 executes after data becomes available")
```

```
print("\nSPECULATIVE EXECUTION")
```

```
print("-----")
```

```
branch_prediction = "TAKEN"
```

```
print("Branch Prediction:", branch_prediction)
```

```
print("Instructions on predicted path are executed speculatively")
```

```
print("\nSMT / HYPER-THREADING")
```

```
print("-----")
```

```
print("Thread 1 -> Uses processor resources")
```

```
print("Thread 2 -> Uses processor resources")
```

```
print("Both threads share processor resources")
```

```
print("\nPERFORMANCE")
```

```
print("Multiple instructions and threads can progress in parallel.")
```

```
SUPERSCALAR EXECUTION
-----
I2 -> SUB executed
I3 -> AND executed
I4 -> OR executed

OUT-OF-ORDER EXECUTION
-----
I1 is waiting for data
I2 is ready -> Execute I2
I3 is ready -> Execute I3
I1 executes after data becomes available

SPECULATIVE EXECUTION
-----
Branch Prediction: TAKEN
Instructions on predicted path are executed speculatively

SMT / HYPER-THREADING
-----
Thread 1 -> Uses processor resources
Thread 2 -> Uses processor resources
Both threads share processor resources

PERFORMANCE
Multiple instructions and threads can progress in parallel.

...Program finished with exit code 0
Press ENTER to exit console.
```

Result

The operation of superscalar, out-of-order and speculative execution was analyzed along with hyper-threading and SMT. These techniques allow multiple instructions or threads to make progress in parallel and can improve instruction throughput and processor resource utilization.